Starry hacks — l'empoisonnement de la chaîne d'approvisionnement

Starpwn CTF 2026 · DEF CON 34 · Aerospace Village

Catégorie : Supply chain **RÉSOLU — 16 août 2026**

`TM|EVENT|COMMAND_ACK|EXFIL|STARPWN{...}`

La mission : cuisiner une mise à jour

« Un de vos démons a trouvé une pile de contrôle orbital bas-de-gamme qui parle à son oiseau en clair. Vous avez une archive du dépôt du logiciel de vol, une cible en ligne, et une chaîne d'approvisionnement qui a l'air juste assez molle. Cuisinez une mise à jour de paquet qui plie le workflow en votre faveur. »

L'objectif n'est pas d'exploiter un bug mémoire ni de casser un chiffrement. C'est d'exécuter du code arbitraire sur le satellite en abusant de la façon dont son logiciel de vol récupère ses **dépendances**. Aucun débordement de tampon : juste de la logique de chaîne d'approvisionnement.

Pourquoi forcer la porte quand on peut empoisonner l'épicerie ? (l'attaque supply chain)

Concept — Quand le maillon faible est une dépendance. Un logiciel moderne n'est presque jamais écrit entièrement à la main : il **importe** des bibliothèques externes écrites par d'autres — pour lire du JSON, parler au réseau, piloter du matériel. Ces bibliothèques sont téléchargées automatiquement depuis des dépôts publics (PyPI pour Python, npm pour JavaScript) au moment de l'installation. D'où l'attaque dite « supply chain » (chaîne d'approvisionnement) : plutôt que de forcer la porte de la maison, on empoisonne l'épicerie où elle fait ses courses. Si un attaquant parvient à publier une version piégée d'une bibliothèque, son code s'exécutera à l'intérieur de toutes les applications qui l'installent, avec tous leurs privilèges. C'est l'attaque qui a frappé SolarWinds en 2020 et des dizaines de paquets npm et PyPI réels depuis.

Ici, le logiciel de vol délègue le traitement des commandes à une dépendance : `from cubesat_upstream_driver import handle_command`. Et son `requirements.txt` — la liste de courses de l'application, qui énumère les bibliothèques à installer — demande `cubesat-upstream-driver >= 1.0.0`. Ce `>=` est la faille : il signifie « n'importe quelle version à partir de la 1.0.0 ». Publier une version **99.0.2**, et le gestionnaire de paquets la choisira automatiquement : pour lui, plus grand = plus récent = mieux. C'est elle qui sera installée à la place du vrai driver.

Comment le butin ressort-il ? (reconnaissance du canal)

L'archive du dépôt tient en trois fichiers. Le cœur est `main.py`, qui montre exactement comment les données pourront ressortir du satellite :

```
from cubesat_upstream_driver import handle_command

def process_command(cmd: str) -&gt; str:
    cmd = (cmd or "").strip()
    if not cmd:
        return "TM|WARN|EMPTY_COMMAND"
```

```
    driver_reply = handle_command(cmd)      # &lt;- notre futur code
    if driver_reply:
        return f"TM|EVENT|COMMAND_ACK|{driver_reply}"   # &lt;- notre canal

    if cmd == "PING":
        return "TM|HEALTH|OK"
    if cmd == "STATUS":
        return "TM|BUS|NOMINAL"
    return f"TM|CMD|IGNORED|{cmd}"
```

Lecture ligne par ligne : quand la station sol envoie une commande au satellite, `main.py` la passe d'abord à `handle_command` — la fonction de la dépendance, donc bientôt notre fonction. Et tout ce que cette fonction retourne est réémis dans la **télémétrie** du satellite, dans un message de type COMMAND_ACK (« accusé de réception de commande », le message par lequel le satellite confirme au sol ce qu'il vient de faire). C'est le canal d'exfiltration rêvé : légitime, attendu, écouté par la station sol. Il suffit que le driver malveillant lise le flag et le renvoie. Autre détail qui compte : le `handle_command` est appelé **avant** les commandes internes comme STATUS — donc même une banale commande STATUS déclenchera notre code.

Un driver qui fait semblant de conduire (l'arme, en deux temps)

Le paquet `cubesat-upstream-driver` version 99.0.2 embarque un `handle_command` qui ne pilote plus le satellite mais cherche le flag et l'exfiltre. Un point de méthode important : l'emplacement du flag sur la cible est **inconnu**. Écrire un driver qui essaie un seul chemin, c'est risquer un aller-retour satellite par supposition. Le driver est donc écrit pour se rabattre sur de la reconnaissance si le flag n'est pas trouvé d'emblée :

```
def handle_command(cmd):
    loot = _loot()          # essaie /flag, /app/flag.txt, variables FLAG...
    if loot:
        return f"EXFIL;{loot}"          # trouve -&gt; on exfiltre direct
    return _recon()         # sinon -&gt; liste les repertoires interessants
                            #         (/, /app, /var/lib/challenge, ...)
```

La version « recon » est la clé pour ne pas tâtonner à l'aveugle : si le premier essai ne trouve pas le flag, la télémétrie renvoie la liste des répertoires du satellite, ce qui indique précisément où chercher au coup suivant. Un essai de reconnaissance coûte une fenêtre de liaison (voir plus bas) — autant qu'il rapporte un maximum d'information.

Le plan, de bout en bout

Le chemin complet, de bout en bout : la contrainte `>=` dans `requirements.txt` ouvre la voie ; le **wheel** v99 est publié (un wheel est le format de paquet binaire standard de Python — ce que pip installe réellement) ; la gateway l'installe automatiquement au prochain build du logiciel de vol ; on attend une fenêtre de liaison ; une commande STATUS banale déclenche l'exécution du code piégé sur le satellite ; le flag redescend dans la télémétrie COMMAND_ACK. Aucun exploit mémoire, aucune cryptographie cassée : uniquement de la confiance mal placée.



Figure 1 — La chaine complete : la contrainte >= (rouge) ouvre la voie ; le wheel v99 est installe au build, puis une commande STATUS pendant une fenetre de liaison declenche l exfiltration.

Les quatre pièges — et leur facture (~1 h 20)

C'est ici que s'est passée l'essentiel des 2 h 30 : environ 1 h 20 perdues sur quatre pièges, dans l'ordre.

1. Envoyer un module `.py` au lieu d'un package (~40 min). Premier réflexe : écrire un seul fichier `cubesat_upstream_driver.py` et l'envoyer au build. Échec, avec un « No such file... cubesat_upstream_driver » et un HTTP 500 sec (le code d'erreur générique « plantage serveur »). Quarante minutes à tester des variantes de nommage et de chemins avant de lire vraiment l'erreur. Explication : en Python, il y a deux façons de distribuer du code — un **module** (un seul fichier `.py`) ou un **package** (un dossier contenant un fichier `__init__.py`, qui peut regrouper plusieurs modules). Le vrai driver est un package ; l'installeur le cherche sous forme de dossier et ne trouve pas le fichier unique. Le message d'erreur disait exactement quelle structure produire depuis le début. Reconstruit en package, le build passe : HTTP 200 (« OK »).

2. Cliquer RESET après l'installation (~15 min). L'interface Mission Control propose un bouton RESET. Réflexe naturel après un build : repartir propre. Erreur — RESET restaure l'environnement virtuel d'origine, donc **efface l'empoisonnement** en réinstallant le vrai driver. Un quart d'heure à se demander pourquoi le driver 99.0.2 avait « disparu ». Rebuild, réinstallation, et cette fois on ne touche plus à rien.

3. Tester hors fenêtre de liaison (~25 min). Premiers essais de la commande STATUS : `SAT_LINK=0 / FLIGHT_UNREACHABLE`. Le satellite refuse de répondre. Pourquoi ? Un satellite en **orbite basse** (LEO, quelques centaines de km d'altitude) tourne autour de la Terre en ~90 minutes et n'est visible de la station sol que quelques minutes par passage — c'est la **fenêtre de liaison**. Hors fenêtre, aucun signal ne peut physiquement monter : le satellite est de l'autre côté de la planète. La vue orbitale de l'interface le montre (`link: true/false`). Vingt-cinq minutes à relancer la commande en boucle en espérant, avant de comprendre que la vue orbitale donne le compte à rebours. Finalement, il a fallu attendre **11 minutes** le passage suivant — exactement comme un vrai opérateur de station sol — avant que la commande parte. C'est le moment le plus réaliste du challenge.

4. Chercher le flag à l'aveugle (évité de justesse). La version « recon » du driver exfiltre d'abord la liste des répertoires, ce qui localise le flag avant de tenter de le lire. Sans elle, chaque essai aurait coûté une fenêtre de liaison complète — plusieurs minutes d'attente par supposition.

La morale de l'histoire

- **La confiance implicite dans les dépendances est une surface d'attaque** : une contrainte `>=1.0.0`, un endpoint d'upload d'artefacts et un build automatique forment ensemble une voie d'exécution de code à distance — sans le moindre exploit mémoire.
- **Les messages d'erreur sont des indices** : la structure de package attendue était écrite noir sur blanc dans l'erreur du build. Lire l'erreur, c'est se faire dicter la correction — et 40 minutes auraient pu être économisées.
- **Le réalisme spatial fait partie du puzzle** : la fenêtre de liaison n'est pas un décor, un satellite LEO n'écoute que quelques minutes par passage. Le timing de la commande compte autant que son contenu.

Flag exfiltré via la télémétrie COMMAND_ACK après installation du driver v99.0.2 et envoi de la commande pendant une fenêtre de liaison. Note : préfixe du flag en capitales, `STARPWN{...}`. Le driver malveillant est dans le dépôt : `github.com/ShymusQuantum/starpwn-2026` (`starry_hacks/`).